

Chapter 2: Objects and Handles

Những phần cần học để phục vụ Reverse Engineering và Malware Analysis

Nguồn: chap2(1).pdf | 45 trang PDF | Số trang in trong sách: 42-86

Kết luận nhanh: Đọc kỹ phần lõi ở trang 42-58 và 62-78. Trong các ví dụ, chỉ giữ lại pattern WinAPI và semantics của object/handle; không cần học WTL, code giao diện hay template C++ ngay lúc này.

1. Bản đồ đọc theo mức độ ưu tiên

Số đầu là trang in trong sách; số trong ngoặc là số trang của file PDF để m nhảy tới nhanh.

Mức độ	Trang sách	Phần cần lấy
BẮT BUỘC	42-45 (PDF 1-4)	Kernel object, Object Manager, reference count, handle là cơ chế truy cập gián tiếp; phân biệt NULL và INVALID_HANDLE_VALUE.
BẮT BUỘC	45-48 (PDF 4-7)	Pattern single-instance bằng named mutex: CreateMutex + GetLastError == ERROR_ALREADY_EXISTS. Chỉ cần hiểu mutex và nhánh kiểm tra; bỏ phần WTL/UI.
BẮT BUỘC	49-58 (PDF 8-17)	Handle table, access mask, flags, Process Explorer, pseudo handles. Đây là phần quan trọng nhất khi đọc WinAPI trong disassembly.
ĐỌC KỸ	62-68 (PDF 21-27)	SECURITY_ATTRIBUTES, binheritHandle, tên object, session namespace, Global\, BaseNamedObjects và các loại object có thể đặt tên.
ĐỌC KỸ	68-78 (PDF 27-37)	Ba cách chia sẻ object; shared memory theo tên; DuplicateHandle và ý nghĩa handle khác số nhưng cùng trỏ tới một object.
ĐỌC LƯỚI	58-62 (PDF 17-21)	RAII và WIL: hiểu mục đích chống handle leak; không cần học template/wrapper lúc này.
ĐỌC LƯỚI	78-84 (PDF 37-43)	Private Object Namespace: hiểu lý do tồn tại và chuỗi API tổng quát; không cần nhớ SID hay code wrapper.
CÓ THỂ BỎ	84-86 (PDF 43-45)	User object và GDI object. Chỉ ghi nhớ chúng không có semantics giống kernel object; quay lại khi phân tích GUI/hook.

2. Phải hiểu được gì sau khi đọc?

- Kernel object nằm trong kernel space; user-mode không cầm con trỏ trực tiếp mà truy cập qua handle.
- Handle value chỉ có ý nghĩa trong handle table của process sở hữu nó. Cùng một số handle ở hai process có thể trỏ tới hai object khác nhau.
- Handle thật thường là bội số của 4 và 0 không hợp lệ; tuy nhiên phải kiểm tra quy ước lỗi riêng của từng API.
- Một object có thể tồn tại sau khi một handle đã đóng vì vẫn còn handle/pointer reference khác. Object chỉ bị hủy khi reference count về 0.

- API Create* có tên có thể trả về handle tới object đã tồn tại. Phải xem GetLastError và ERROR_ALREADY_EXISTS.
- Không phải API thất bại nào cũng trả NULL: CreateFile thất bại trả INVALID_HANDLE_VALUE (-1).
- Access mask thể hiện quyền của chính handle đó, không chỉ quyền chung của process. OpenProcess xin quyền gì thường hé lộ ý định của code.
- Pseudo handle như GetCurrentProcess() trả giá trị đặc biệt âm; không được CloseHandle như handle thật.
- Named object nằm trong Object Manager namespace và thường có prefix theo session; Global\ đưa object vào namespace dùng chung giữa session.
- Chia sẻ object có ba đường: theo tên, kế thừa handle khi tạo process con, hoặc DuplicateHandle.
- Hai handle có giá trị khác nhau vẫn có thể trỏ tới cùng kernel object; Object Address trong Process Explorer giúp kiểm chứng.

3. Liên hệ trực tiếp với REV và MA

3.1 Named mutex: dấu vết single-instance

Pattern cần nhận ra: CreateMutex(..., name) rồi kiểm tra GetLastError() == ERROR_ALREADY_EXISTS. Malware thường dùng mutex để không chạy trùng, phối hợp nhiều thành phần hoặc làm dấu hiệu môi trường. Tên mutex có thể trở thành IOC, nhưng một mutex tự nó không đủ để kết luận độc hại.

3.2 Access mask: đọc ý định thay vì chỉ đọc tên API

Khi thấy OpenProcess, hãy tìm đối số dwDesiredAccess. Ví dụ của chương dùng PROCESS_TERMINATE. Trong các chương sau, các quyền truy cập bộ nhớ, tạo thread hoặc duplicate handle sẽ giúp phân biệt truy vấn thông tin, thao tác bộ nhớ và chuỗi hành vi giống process injection.

Cảnh báo phân tích: Một access mask mạnh chỉ là tín hiệu về khả năng; phải ghép với API gọi tiếp theo và dữ liệu đích trước khi kết luận.

3.3 Object name và namespace: artefact để hunting

Mutex, Event, Section, Job, Timer và một số object khác có thể đặt tên. Khi phân tích mẫu, hãy ghi lại:

- Tên gốc được truyền vào Create*/Open*.
- Prefix theo session hay Global\; dùng nhằm tên user-mode với đường dẫn đầy đủ trong Object Manager.
- Process nào đang giữ handle, type gì, access mask nào và object address có trùng không.

3.4 Shared memory và DuplicateHandle: IPC và chia sẻ tài nguyên

Chuỗi CreateFileMapping → MapViewOfFile → UnmapViewOfFile biểu diễn một vùng nhớ dùng chung. Trong malware, vùng này có thể chứa cấu hình, trạng thái, dữ liệu hoặc payload giữa các process. DuplicateHandle tạo một handle hợp lệ trong process đích; giá trị mới có thể khác nhưng trỏ tới cùng object. Khi reverse, luôn xác định handle đang hợp lệ trong context process nào.

3.5 Private namespace: biết để không bị bất ngờ

Chỉ cần hiểu rằng nhiều process hợp tác có thể dùng named object nhưng giới hạn namespace bằng boundary descriptor/SID. Công cụ có thể không hiển thị đầy đủ tên. Đây là khái niệm hữu ích khi một mẫu có IPC nhưng artefact tên object không lộ theo cách quen thuộc; chưa cần học thuộc toàn bộ API.

4. API và khái niệm nên nhận diện bằng mắt

API / khái niệm	Khi REV/MA cần nghĩ tới
CreateMutex / OpenMutex	Mutex đặt tên; dấu vết single-instance và đồng bộ giữa process.
GetLastError	Phân biệt Create* vừa tạo object mới hay chỉ mở object đã tồn tại.
OpenProcess	dwDesiredAccess cho biết code muốn làm gì với process đích.
CloseHandle	Kết thúc reference của handle; giúp nhận diện cleanup và handle leak.
GetCurrentProcess / GetCurrentThread	Trả pseudo handle; giá trị âm không phải lỗi.
SetHandleInformation	Bật/tắt inherit hoặc protect-from-close trên handle.
CreateFileMapping / MapViewOfFile	Shared memory/section object; IPC hoặc vùng dữ liệu dùng chung.
DuplicateHandle	Tạo entry trong handle table của process đích, cùng trỏ tới kernel object.
Create/OpenPrivateNamespace	Object có tên nhưng namespace được giới hạn giữa các process hợp tác.

5. Những đoạn có thể bỏ qua mà không mất nền tảng

- Trang 46-48: phần FindWindow, PostMessage, message map và CListBox của ví dụ SingleInstance. Giữ lại CreateMutex và ERROR_ALREADY_EXISTS.
- Trang 58-62: chi tiết triển khai move constructor, template và cài WIL/NuGet. Chỉ hiểu RAII giúp CloseHandle tự động và ngăn handle leak.
- Trang 68-73: code WTL lấy text từ edit box. Giữ lại CreateFileMapping, MapViewOfFile, access FILE_MAP_READ/WRITE và UnmapViewOfFile.
- Trang 82-84: chi tiết tạo SID và tự viết WIL wrapper cho private namespace. Chỉ cần nhớ flow khái niệm.
- Trang 84-86: chi tiết User/GDI objects nếu hiện tại m tập trung process, memory, injection và malware behavior.

6. Bài thực hành tối thiểu

1. Mở Process Explorer, bật Lower Pane → Handles và Show unnamed handles and mappings. Chọn một process quen thuộc, đọc Type, Handle, Object Address, Access và Decoded Access.

2. Mở WinObj bằng quyền Administrator; xem \ObjectTypes, \BaseNamedObjects và \Sessions\<id>\BaseNamedObjects. Đối chiếu tên object với Process Explorer.
3. Viết console program tạo named mutex. Chạy hai instance và in kết quả ERROR_ALREADY_EXISTS. Đây là bài code quan trọng nhất của chương.
4. Tùy chọn: chạy hai process dùng cùng named file-mapping; xác nhận handle value khác nhau nhưng Object Address giống nhau.
5. Tùy chọn nâng cao: thử DuplicateHandle trong cùng process với access mask hẹp hơn và quan sát hai entry trong Process Explorer.

7. Checklist khi gặp HANDLE trong disassembly

- Handle này được tạo/mở bởi API nào?
- Giá trị lỗi của API là NULL hay INVALID_HANDLE_VALUE? Code kiểm tra đúng không?
- Object type là Process, Thread, File, Section, Mutex, Event, Token hay loại khác?
- Desired access mask là gì? Những API gọi sau có dùng đúng quyền đó không?
- Object có tên không? Tên thuộc session, Global\ hay private namespace?
- Handle có được kế thừa, duplicate hoặc truyền qua IPC không?
- Handle value đang được nhìn từ process nào?
- Cuối đường đi có CloseHandle hoặc cleanup tương đương không?

Mốc hoàn thành chương: M không cần nhớ mọi prototype. Chỉ cần nhìn một chuỗi WinAPI và giải thích được object nào đang được thao tác, handle thuộc process nào, quyền của handle, vòng đời object và cách object được chia sẻ.

8. Thứ tự học đề xuất

1. Lượt 1 - 25 đến 35 phút: đọc trang 42-45, 49-58, 62-68; bỏ code UI.
2. Lượt 2 - 20 đến 30 phút: đọc trang 68-78; tập trung sharing by name và DuplicateHandle.
3. Lượt 3 - 15 phút: dùng Process Explorer + WinObj để biến khái niệm thành hình ảnh thực tế.
4. Sau đó: lượt 58-62 và 78-86; chỉ quay lại chi tiết khi code hoặc mẫu malware thật sự dùng chúng.